

WHEN THE PILOT AUDITS THE TOOL: TWO VERIFICATION BLIND SPOTS EXPOSED AND CLOSED BY A REAL COMPUTATIONAL-MATHEMATICS PROOF

ABSTRACT. Proof-orchestration systems promise that a machine-checked release gate makes a computational proof trustworthy. But a gate only checks the properties it was told to check. We report a case in which a genuine research proof — the FP-0.35 conjecture on positivity of the Weil quadratic form at $L = 7/20$ — acted as an adversarial pilot against its own verification layer, `proofctl`, and exposed two *kernel blind spots* that every prior release had silently passed: (i) a generator that merely *copied* a stored certificate while its attestation claimed from-scratch recomputation, and (ii) a checker that recomputed honestly but *omitted a term* of the quantity it claimed to verify, emitting a false pass with a $16\times$ -inflated margin. Neither is caught by digest-binding, schema validation, or “all obligations pass”: the copy-generator produces byte-identical replays, and the omitted-term checker is internally consistent. We describe the two release conditions added in response — C10 (no copy-only generator) and C11 (checker mutation coverage) — the epistemic discipline (a written “proof constitution”) that the same incident forced, and the closeout in which the corrected pilot was driven back through the hardened gate. A second, independent instance of the same blind-spot class (a dependency lemma whose checker also lacked mutation coverage) surfaced during closeout, confirming the conditions generalise beyond the originating bug. The broader lesson is methodological: the highest-value feedback a real proof can give a verification system is not “it verified” but “here is a true statement your gate accepted for the wrong reason.”

1. INTRODUCTION

Computational proofs increasingly combine heterogeneous evidence: formal proofs in proof assistants, certified interval arithmetic, and independently replayable numerical certificates. A proof-orchestration layer stitches these into one artefact and gates its release on machine-checkable conditions. `proofctl` is such a layer: an open-source Go tool that binds every attestation to the exact digests of its checker, schema, and dependencies, derives claim acceptance solely from a checker’s `obligation_results` (never from a self-reported `outcome` field), and blocks release until a set of conditions C01–C11 all pass against the full attestation graph.

The comfortable assumption is that once such a gate is green, the computational proof is sound up to the mathematics of the individual checkers. This paper is about the gap in that assumption. A green gate certifies only what its conditions encode; a property nobody thought to encode can be violated by every release and never noticed.

Our evidence is concrete. During the development of a real research proof — FP-0.35, the assertion that the local Weil quadratic form on $L^2(-7/20, 7/20)$ has strictly positive infimum [1] — the proof’s own certification exposed two such properties, each violated by a certificate that `proofctl` had been happily accepting:

- **Copy-as-recomputation.** The main certificate’s “generator” was a one-line file copy, `python3 -c "shutil.copy('certs/thm-fp-035.json', {cert})"` tagged `replay_mode = from_scratch`. Every replay reproduced the stored bytes exactly, so digest-comparison passed. The attestation claimed the result was recomputed; it was transcribed.
- **Omitted-term checker.** The checker reported all obligations `pass` while computing the second-moment matrix $S^{(0)}$ as its S_{KK} block alone, silently dropping the three positive blocks $S_{VV} + S_{VK} + S_{KV}$. Dropping positive mass shrinks the Schur residual and *inflates* the certified eigenvalue — here by roughly $16\times$ (0.01494 reported vs. 0.00095 true). The checker was internally consistent and passed its own schema; it was simply verifying a different, easier quantity than the one it claimed.

Crucially, and this is the subtlety that makes the case worth reporting, the *conclusion* was correct throughout: $\lambda(7/20) > 0$ holds; both defects are process defects, not sign errors. A verification system that only ever asks “is the answer right?” would see nothing wrong. The value of the pilot was to force the question “was the answer obtained the way the attestation says it was?” — and the answer was no, twice.

The remainder of the paper describes `proofctl`’s architecture (Section 2), the FP-0.35 proof graph it orchestrates (Section 3), the two blind spots and the conditions C10/C11 added to close them (Section 4), the mutation-catalog design that operationalises C11 together with a nuance that naive mutation testing gets wrong (Section 5), the epistemic discipline the incident forced (Section 6), and the closeout in which the corrected pilot passed the hardened gate and a second instance of the blind-spot class was found and fixed (Section 7).

2. ARCHITECTURE OF `PROOFCTL`

2.1. ProofGraph and claims. A *ProofGraph* is a directed acyclic graph of *claims*. Each claim carries a `statement_digest` (SHA-256 of its text), a list of `dependencies` with required assurance levels, a `checker_policy` naming the checker that verifies it, and an `obligations` list specifying exactly what that checker must certify.

2.2. Checker protocol. A checker is a subprocess that receives a certificate path and emits JSON:

```
{ "protocol_version": 2,
  "obligation_results": [ {"id": "...", "verdict": "pass"}, ... ],
  "metadata": { "format_version": "...", ... } }
```

with exit codes 0 (certified), 1 (uncertified), 2 (malformed), 3 (resource/blocked). Acceptance is *derived* by `proofctl` from the verdict set; a checker cannot assert its own acceptance. The obligation set is matched *exactly* against the contract: a missing, extra, or duplicated obligation is a hard reject, so a checker cannot quietly shrink what it is answerable for.

2.3. Identity closure and staleness. Every attestation binds the full identity closure: statement digest, dependency digests, checker digest, and schema digest, recomputed and re-verified on load. Any change to an input automatically marks downstream attestations *stale*, forcing re-verification rather than trusting a cached verdict.

2.4. Release gate. The gate evaluates conditions C01–C11 against the whole graph. Representatives: C01 every claim accepted (derived from obligation results, not from a writable `outcome`); C04 replay consistency; C09 no development-only (`native`) runtime on a releasable claim. The two conditions this paper is about, C10 and C11, are latecomers: they exist because the pilot showed that C01–C09 were jointly insufficient.

3. THE PILOT: FP-0.35

FP-0.35 asserts $\lambda(7/20) := \inf_{v \neq 0} Q_W^{7/20}(v)/\|v\|^2 > 0$, a finite-scale positivity statement (it does *not* imply the Riemann Hypothesis). The proof reduces, via a split-residual Schur criterion, to positive definiteness of a matrix $C = b_L F - R_\eta$ in each of two parity sectors, where R_η is built from the full second moment $S^{(0)} = S_{VV} + S_{VK} + S_{KV} + S_{KK}$, the certified Weil constant $c_L \approx 1.36527$, and prime-shift matrices. The orchestrated proof graph has 17 inter-dependent claims spanning three evidence regimes: pure rational arithmetic (window and absorption lemmas), Lean 4 formalisation of the integer skeleton, and 256-bit Arb interval arithmetic for the archimedean integrals.

This heterogeneity is why orchestration is needed — and why the blind spots matter. The main claim depends on two sector lemmas, each of which depends on an archimedean base certificate of 2.4 MB whose re-verification takes tens of minutes. Nobody re-derives such a chain by hand; the entire point of the gate is to make its acceptance trustworthy without doing so.

4. TWO BLIND SPOTS

4.1. **C10: copy masquerading as recomputation.**

Principle 4.1 (Recomputation must recompute). An attestation that claims from-scratch recomputation must run a generator that performs computation, not a file copy. A byte-exact replay of a copy proves only that copying is deterministic.

The failure mode is invisible to digest binding precisely because it is *too* reproducible. `proofctl`’s replay compares the generated certificate’s digest against the stored evidence; a `shutil.copy` reproduces the stored bytes perfectly, so the comparison always passes. The attestation’s `replay_mode = from_scratch` is a self-report the gate had been trusting.

C10 closes this by inspecting the recorded `generator_cmds` of every attestation that *claims* substantive recomputation (`replay_mode = from_scratch`, or a reproducible/exact-replay assurance) and rejecting it when the generator is purely a copy/no-op primitive (`shutil.copy`, `cp`, `cat`, `ln`, ...) with no sign of computation. The check is deliberately conservative — a generator that imports `shutil` and calls `recompute/integrate/schur` is not flagged — so it blocks the fraud pattern without penalising legitimate generators that happen to touch the filesystem.

4.2. **C11: honest recomputation of the wrong quantity.** C10 is necessary but not sufficient. A checker can genuinely compute — no copy anywhere — and still emit a false pass by computing an *incomplete* version of the asserted quantity. That is exactly the omitted-term bug: the generator computed $S^{(0)} = S_{KK}$ only, a real computation, dropping the three other positive second-moment blocks. The result is a smaller residual, a larger (inflated) minimum eigenvalue, and a confidently-passed obligation.

Principle 4.2 (Sensitivity is verification). A checker that claims to verify a quantity assembled from terms $T_1, \dots, T_m$ must be demonstrably sensitive to each T_i : perturbing T_i must be capable of changing the verdict. A term whose perturbation never moves the verdict is a term the checker does not actually check.

C11 operationalises this: any attestation claiming substantive recomputation must carry, in its metadata, `mutation_kill_rate = "100%"` and a non-empty `mutation_catalog_digest` pinning which mutants were run. The kill rate must be exactly 100%: a checker that lets any catalogued mutant survive has a blind spot of the same class as the omitted-term bug. C11 cannot be satisfied by a certificate-reading checker that trusts supplied numbers; it forces a recompute-style checker whose verdict genuinely depends on every asserted term.

Remark 4.3 (Why digests and schemas miss both). Digest binding proves the same bytes were checked by the same script; it says nothing about whether the script computes the claimed quantity. Fail-closed schema validation rejects unknown or malformed fields; both defective certificates were schema-valid. “All obligations pass” is exactly the output a hard-coded or omitted-term checker produces. The two blind spots live in the space between “the artefact is well-formed and stably reproduced” and “the artefact means what it claims.”

5. MUTATION CATALOGS, AND A NUANCE

To satisfy C11, each recompute checker ships a *mutation catalog*: a list of single-term perturbations of the assembled matrices, each re-judged by the *same* assembly and positivity judge the checker uses (no duplicated logic), recording whether the perturbation “kills” the verdict. For FP-0.35 the catalog has six mutants per sector — e.g. zero the second moment $S^{(0)}$ (the retired-bug class), flip the sign of the coupling diagonal, remove the kinetic coercivity, scale a prime-coupling block — and achieves 6/6 kills.

A subtlety, easy to get wrong, deserves emphasis, because getting it wrong reintroduces the very failure C11 guards against:

Principle 5.1 (Kill criterion must match a term’s true influence). A mutant that zeros a term whose genuine influence is negligible will not flip the verdict — and this does *not* mean the checker is blind to that term. Classify mutants. *Judge-sensitivity* mutants (sign flips, zeroing a

dominant term, swapping the positivity judge) must flip the verdict. But a term with $\sim 10^{-4}$ influence, zeroed, should be probed by a *large-factor scaling* mutant that provably moves the pivot, or by a relative-tolerance threshold — not by demanding a sign flip it cannot cause.

Concretely, in the even sector zeroing the prime self-second-moment $S^{(2)}$ moves the minimum pivot from +0.00870 to +0.00871 — a $\sim 10^{-4}$ change. A naive “100% kill or the checker is blind” rule would wrongly flag the checker as insensitive to $S^{(2)}$. In truth $S^{(2)}$ barely influences the even sector; the correct probe is a 100× scaling mutant, which drives the pivot sharply negative and is cleanly killed. The catalog therefore separates *judge* mutants (must flip) from *recompute-verification* mutants (must move measurably), each scored by an appropriate criterion.

6. THE DISCIPLINE THE INCIDENT FORCED

The two blind spots were the tool’s failure. But the same night exposed a *human* failure mode that no gate can catch, and that we found worth codifying as an executable-where-possible “proof constitution.” We record the parts most relevant to anyone building or trusting such a system.

Principle 6.1 (Narrative resistance). A computed number that supports a dramatic story — “we found a fatal bug,” “the proof fails” — demands *more* scrutiny, not less. Excitement is a bias multiplier, not evidence.

During diagnosis, an ad-hoc script produced a minimum pivot of -0.043 (“FP-0.35 fails!”), which was accepted three times because it fit the exciting “we found a fatal bug” narrative. It was a bug in that script’s residual assembly; the true value was $+0.0087$ all along, and downstream conclusions had to be retracted. The corrective was mechanical, not attitudinal:

Principle 6.2 (Diff artefacts before narrating a disagreement). When two implementations give different scalars, dump and diff the full intermediate *matrices* element-wise to locate the exact divergence before explaining “why.” A scalar mismatch plus a plausible story is not a diagnosis.

The element-wise comparison of the two independent Schur assemblies settled it in one step: the matrices agreed to $\max |C_A - C_B| = 4 \times 10^{-3}$, both yielding $+0.0087$; the -0.043 existed in no correct computation.

Principle 6.3 (Process defect $\neq$ wrong conclusion). A defective process (copy generation, omitted term, hard-coded verdict) does not automatically make the conclusion wrong. State precisely which is broken.

This principle is what let the pilot be both a genuine bug report against the tool and a preserved-correct mathematical result: the certificate’s process was doubly broken, yet $\lambda(7/20) > 0$ was never in question. Conflating the two would have either discarded a true theorem or excused a broken pipeline; keeping them separate is what made C10/C11 the right response rather than a retraction.

A last principle bears directly on why C11 takes the form it does:

Principle 6.4 (Prefer recomputation over certificate-reading). A checker that recomputes the claim from primitives is structurally immune to copy-certificate and self-attestation attacks, unlike one that reads and validates a supplied certificate. Recompute checkers still need mutation coverage for omitted-term bugs — but they cannot be fooled by a doctored certificate.

7. CLOSEOUT, AND A SECOND INSTANCE

Hardening a gate is only credible if the originating artefact is then driven through it. We regenerated the FP-0.35 certification cleanly: the main certificate was produced by a genuine four-term recomputation (satisfying C10), and the checker was made to emit mutation-catalog metadata with a 100% kill rate and a non-empty catalog digest (satisfying C11). The release dry-run then passed all thirteen conditions with no blockers, and all 17 claims reached the accepted state.

The closeout also produced the paper’s sharpest evidence that the conditions capture a *class* of defect, not a single bug. C11 fires on every attestation that claims recomputation — and

`proofctl`'s replay tags every generated attestation `from_scratch`. The two sector-lemma attestations (the main claim's dependencies) therefore also required mutation coverage, but their checker — a distinct engine that replays the archimedean base and runs an exact-rational LDL^T — emitted no mutation metadata at all. The originating fix had addressed only the main claim; the gate correctly refused to go green until the dependency checkers were given their own honest mutation catalogs (again 6/6 kills per sector), built by refactoring the checker's assembly and judge into shared functions so that the catalog perturbs exactly what the checker asserts, with no duplicated logic. That a second, independent instance of the omitted-term blind-spot class surfaced — and was blocked — during routine closeout is the strongest available evidence that C11 encodes a general property rather than patching one certificate.

Remark 7.1 (What `proofctl` still does not guarantee). The gate verifies that the pinned checker ran, that evidence digests match, that attestations are untampered, and that obligation and mutation-coverage conditions hold. It does *not* verify that a checker's mathematics is correct; that comes from the checker code, its tests, and independent review. Mutation coverage raises the floor — a checker must be sensitive to every asserted term — but a checker can be sensitive to a term and still integrate it with the wrong constant. C10/C11 close two specific, previously-invisible holes; they do not turn orchestration into a formal kernel.

8. RELATED WORK AND DISCUSSION

Formal kernels (Coq, Isabelle, Lean) give the strongest guarantees but demand the full proof term; `proofctl` is complementary, orchestrating *computational* evidence that such systems could in principle certify. Certificate-passing compiler verification (CompCert, CertiCoq) shares the architecture in a different domain. Reproducible-computation tooling (containerised notebooks, workflow managers) achieves byte-reproducibility but not the `proofctl` notions of an exact obligation set, a fail-closed gate, or — as this paper stresses — a check that the reproduced computation is the *claimed* one rather than a copy or a term-incomplete surrogate.

The transferable lesson is about the *direction* of trust between a proof and its verifier. A verification system is usually treated as auditing the proof. The FP-0.35 pilot inverts this: a sufficiently real proof, pushed hard enough, audits the verifier, and the defects it surfaces are exactly the properties the verifier's designers did not think to encode. Copy-as-recomputation and honest-computation-of-the-wrong-quantity are two such properties; there are surely others. The practical recommendation is to treat every genuine pilot as an adversary against the gate, and to regard “a true statement your gate accepted for the wrong reason” as the most valuable bug report a proof can file.

9. CONCLUSION

A green release gate certifies the properties its conditions encode, no more. Two properties — that a recomputation actually recomputes, and that a checker is sensitive to every term it claims to verify — were absent from `proofctl`'s gate and violated by a real proof's own certificate, undetectably to digest binding and schema validation, because the conclusion happened to be correct. Encoding them as C10 and C11, and re-driving the corrected pilot through the hardened gate (where a second instance of the same class was caught), turned a pair of embarrassing near-misses into durable, reusable conditions. The episode is a small instance of a general principle worth stating plainly: the best thing a real computation can do for a verification system is to break it honestly.

REFERENCES

- [1] *Structural lemmas for the first-prime window of the Weil quadratic form*, preprint (2026). doi:10.5281/zenodo.21807498.
- [2] F. Johansson, *Arb: efficient arbitrary-precision midpoint-radius interval arithmetic*, IEEE Trans. Comput. **66** (2017), 1281–1292.
- [3] L. de Moura and S. Ullrich, *The Lean 4 theorem prover and programming language*, CADE 28 (2021), 625–635.
- [4] R. A. DeMillo, R. J. Lipton, and F. G. Sayward, *Hints on test data selection: help for the practicing programmer*, Computer **11** (1978), 34–41. [Origin of mutation testing.]